01 / SYSTEM MAP

Decentralised Lending Service

Marco Pernisco Student ID: 683674
P2P Systems and Blockchains Lab 2025/26 / Trace 1.1



Contributors pool ETH and decide which loans to fund. A trusted oracle records historical Bitcoin liquidity; each approved loan receives its own contract.

Actors and responsibilities

CONTRIBUTOR

Deposits and withdraws ETH; votes; claims interest and compensation. The same user may also be an applicant.

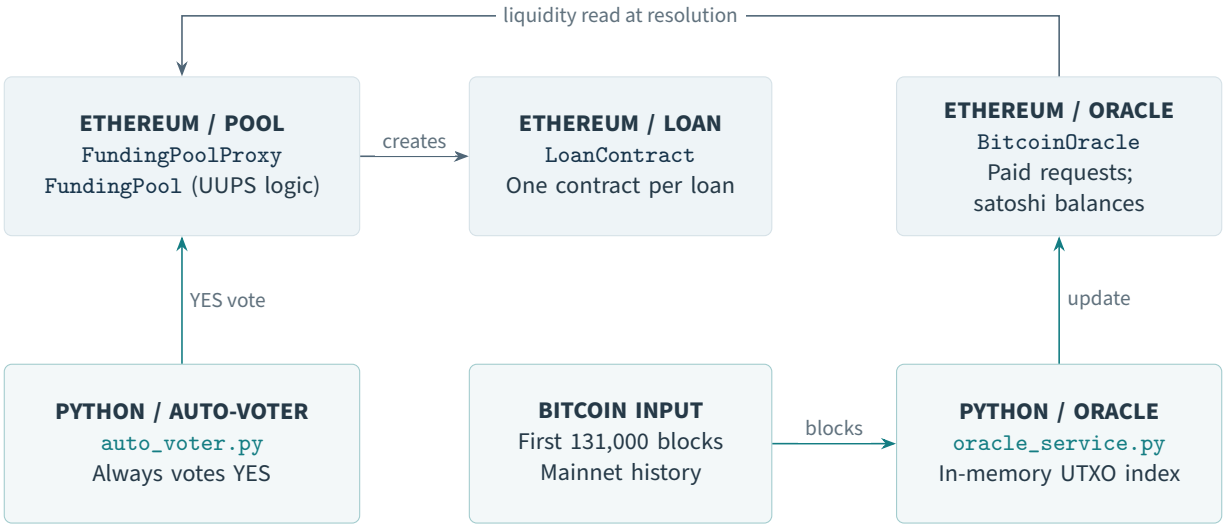
APPLICANT / BORROWER

Proposes loan terms and a BTC address; resolves the proposal; receives ETH and repays the loan.

OWNER / ORACLE OPERATOR

The pool owner controls upgrades and termination. The trusted oracle operator publishes Bitcoin balances.

Contracts and off-chain services



Separate balances inside the pool

Funding pool	Contributor capital: disposable ETH can be withdrawn or loaned; locked positions back outstanding loans.
Compensation pool	Reserve funded by collateral portions of interest, surplus repayments and rounding dust; used to compensate failed loans.
Claimable interest	Contributor gains tracked separately from deposits; paid through `claimInterestGain()`. Not loanable capital.

Didactic trust model. BTC ownership is not proved and BTC are not locked. The pool reads the stored balance at a fixed rate of **1 BTC = 30 ETH**. The proxy holds ETH and state; the implementation supplies executable logic.

02 / PROTOCOL

Lifecycle & accounting

Rules applied by the pool and each dedicated loan contract.

1. Proposal, voting and resolution

Contributor. Deposits must exceed **100,000 wei**. A contributor retains voting eligibility even when all capital is locked. Each vote is YES or NO; abstentions count as NO. At resolution, the pool recomputes weights using each contributor's *current disposable balance*; ties reject.

Applicant. Specifies principal, interest rate (1–100), duration in blocks and a Bitcoin address. Only the original applicant resolves the proposal, after at least **12 blocks**. Voting stays open until resolution. The ABI field `votingDeadlineBlock` denotes the earliest resolution block.

FundingPool. Insufficient disposable ETH or BTC liquidity rejects and closes the proposal. Approval also closes voting and creates a new `LoanContract`. All time periods use block-height differences.

2. Proportional allocation and loan terms

The pool locks each contributor's proportional share of current disposable funds. Integer-division remainders reduce the **actual disbursed principal**; they are not assigned to an arbitrary participant. The loan snapshots the current collateral percentage, which remains fixed for that loan.

3. Installments, principal refunds and interest

Only the original borrower may repay an active or failed loan. Each installment contains both principal and interest. With principal P, rate r, total interest $I = \lfloor Pr/100 \rfloor$ and cumulative accepted payment $C \le P + I$:

$$\underbrace{B(C) = \left\lfloor \frac{CP}{P+I} \right\rfloor}_{\text{cumulative principal}}, \qquad \underbrace{J(C) = C - B(C)}_{\text{cumulative interest}}.$$

Each installment applies the change in these targets. Collateral uses changes in $\lfloor Jc/100 \rfloor$, so splitting a payment preserves the cumulative principal/interest and collateral rounding totals.

- **Principal:** refunds positions from highest to lowest *initial* loan share, breaking ties by descending numerical address. Merge sort computes the order once; pre-upgrade loans without an order are prepared lazily.
- **Interest:** collateral enters the compensation pool; gains are credited separately in proportion to original loan shares and withdrawn using `claimInterestGain()`. Distribution dust also enters compensation.
- **Overpayment:** any amount beyond the entire principal-plus-interest debt enters compensation. A single installment may repay the whole debt.

4. Success, default and compensation

FUNDED → REPAID
Full repayment by the due block is successful. Global collateral decreases by five points. The loan closes; recovered capital becomes disposable again.

OVERDUE → DEFAULTED
Late repayment or a valid default/compensation request records failure once. Global collateral increases by five points. Later repayment cannot reverse it.

Collateral starts at **50%** and is bounded to **1–100%**. Compensation covers unpaid principal only, up to the available reserve. A valid request with an empty reserve records default but pays zero; the claim remains for later refills. Compensation forfeits the same amount of future principal recovery, while original interest weights remain unchanged.

Lifecycle control. Only the owner may upgrade the pool or terminate it when no active loans remain. Termination preserves recovery operations. Existing loan contracts keep their original bytecode after an upgrade. The installment split, minimum voting period and empty-reserve behavior are documented interpretations of the trace's less explicit wording.

03 / VERIFICATION

Tests & gas evaluation

Measured transaction costs, with reproducible build settings.

32	49	20
Contract tests passed	Offline Python tests passed	Operation scenarios profiled
4 Solidity + 28 TypeScript	No real keys or RPC	Measured receipt gasUsed

Environment. Hardhat 3.18.1; TypeScript 6.0.3; `node:test`; viem 2.57.3. Solidity 0.8.24, optimizer 200 runs and EVM **Berlin**; simulated chain ID 202526, matching the supplied genesis.

Operation costs by responsible contract

Operation	Scenario	Gas units
FundingPool / via proxy		
`contribute()`	New contributor	140,923
`withdraw()`	Partial withdrawal	47,933
`withdraw()`	Final withdrawal	32,412
`createLoanProposal()`	New proposal	234,798
`voteOnProposal()`	YES vote	106,936
`executeApprovedLoan()`	Approved	1,049,274
`executeApprovedLoan()`	Rejected	90,860
`claimInterestGain()`	Claim accrued gain	24,691
`requestCompensation()`	Reserve pays claim	52,652
`requestCompensation()`	Reserve empty	43,786
`upgradeToAndCall()`	Compatible V2	37,769
`terminate()`	No active loans	36,295
LoanContract		
`repay()`	Partial installment	168,082
`repay()`	Final installment	90,716
`repay()`	Late repayment	138,782
`markDefaulted()`	Overdue loan	103,758
BitcoinOracle		
`requestUpdate()`	Paid request	25,193
`update()`	First nonzero balance	49,426
`update()`	Existing balance	32,326
`withdrawFees()`	Operator collects fees	30,868

Source: `docs/gas-measurements.json`, generated from transaction receipts including intrinsic gas. One-contributor scenario with separate partial/final repayments. Values are gas units, not wei or universal upper bounds. Getters invoked with `eth_call` incur no paid transaction fee.

> **Oracle update fee.** $49,426 \times 0.1$ gwei $= \mathbf{4,942,600,000,000}$ wei.
> Calibration uses a first nonzero update of a 34-character historical address. Existing-slot updates cost less. Compiler, bytecode or scenario changes require recalibration.

What the checks cover

Protocol: voting boundaries, allocation dust, installments, defaults, compensation retries, termination, access, upgrades and reentrancy. Native Solidity fuzzing checks repayment conservation with **256 generated cases**.

Python: genesis exclusion, invalid/incomplete data, atomic UTXO indexing, event recovery, retries, receipt failures and fresh oracle events. TypeScript strict checks also pass. `npm run test:gas` reproduces profiling; `npm run test:coverage` instruments tests without overwriting gas measurements.

04 / SECURITY

Attacks & operational limits

Required adversarial demonstration and explicit trust assumptions.

Reentrancy: intentionally changing the real pool

The test-only `FundingPoolVulnerable` inherits `FundingPool` and changes withdrawal: it caches the position, transfers ETH first, then writes stale balances. `ReentrancyAttacker` recursively calls `withdraw(amount)` from its receive function.

```
uint256 beforePosition = totalContributed[msg.sender];
uint256 beforePool = totalUnlockedFunds;
(bool sent, ) = payable(msg.sender).call{value: amount}("");
if (!sent) revert TransferFailed();
totalContributed[msg.sender] = beforePosition - amount;
totalUnlockedFunds = beforePool - amount;
```

VULNERABLE VARIANT
The same claim remains available during reentry. A 1 ETH attacker deposit can drain the other contributor's 10 ETH. The test observes an empty victim pool.

NORMAL FUNDING POOL
Accounting changes before the external call. Reentry fails while the legitimate withdrawal finishes. The honest contributor's 10 ETH remain in the pool.

The same attacker is tested against both proxies. Simply moving a checked live subtraction after the transfer might underflow and revert in Solidity 0.8; stale assignments make this modified example actually exploitable. Test variants live in `contracts/test/` and are never deployed by the service setup. These checks do not constitute a general security proof.

Unfair remuneration without reentrancy

Majority contributor: 6 ETH + honest minority: 4 ETH → 10 ETH loan

The majority contributor can approve a loan to a colluding borrower and default. The attacker moves 6 ETH of its own capital but captures the minority's 4 ETH: the net gain is **4 ETH**, not the full loan. Any sufficiently funded BTC address can be cited because the trace requires no ownership proof. Priority refunds also favor large initial positions: with backing 6/4, a base repayment of 6 restores the whale's position before the smaller contributor recovers any principal. These are consequences of the specified economic rules.

Boundaries of the implementation

- **Bitcoin oracle.** Applies blocks in chain order and checks payload hashes/Merkle roots, but trusts historical input rather than implementing full Bitcoin consensus. It excludes the unspendable genesis reward and publishes nothing after failed/incomplete synchronization.
- **Service reliability.** SQLite persists requests and signed transactions before advancing/broadcasting. Services retry temporary failures; checkpoints detect chain replacement and nonce conflicts remain inspectable. The target is a single-node private chain, without automatic reorganization recovery.
- **Scale and upgrades.** Empty contributor positions are removed and refund order is sorted once. Work still grows with contributor count and must fit the block gas limit. V2 tests preserve ownership, state and active positions; previously created loan contracts remain immutable.

Integration verification / 5 November 2026. An isolated Geth 1.13.14 chain used the course genesis, 10-second Clique blocks and a real 131,000-block Bitcoin replay. All 20 receipts succeeded; 16 final checks passed. A 1 ETH loan at 5% was repaid in two 0.525 ETH installments, producing 0.025 ETH each in gain and reserve. A second 0.5 ETH loan defaulted: compensation paid 0.025 ETH, then zero on retry, preserving the remaining 0.475 ETH claim. Selected source hashes and final state are in `docs/default-demo-verification.json`. Earlier evidence in `docs/bitcoin-validation.json` and `docs/geth-demo.json` remains explicitly historical; the user's existing chain and environments were untouched.

05 / USER MANUAL

Run the project

Successful repayment, then default and compensation on the private chain.

1. Install and validate Bitcoin input

Use a supported even-numbered Node release (22.13.0+), Python 3.10+ and Geth **1.13.14** for the supplied Clique genesis. Run from the project root:

```
./setup.sh
./.venv/bin/python python/oracle/check_data.py --data-dir bitcoin --summary
```

`setup.sh` installs dependencies and Geth, creates the Python environment and compiles; deployment is separate. Supply decoded mainnet `blk*.dat`. The checker must report `valid: true`, 131,000 blocks and height 130,999; it lists funded addresses without needing keys or an Ethereum connection.

2. Start the private chain / terminal 1

```
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore --networkid=202526 \
  --http --http.addr=127.0.0.1 --http.port=8545 --http.api=eth,net,web3 \
  --mine --miner.gasprice=100000000 --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

3. Deploy the service / terminal 2, project root

```
./.venv/bin/python python/setup.py
```

The prefunded account **only transfers ETH** to new accounts. A new deployer creates the oracle, implementation and initialized proxy. Setup rebuilds bytecode, validates the chain and deployment, then writes service environments including `DEPLOYMENT_BLOCK`. Each run creates a separate deployment.

4. Start both Python services / separate terminals

```
./.venv/bin/python python/oracle/oracle_service.py
./.venv/bin/python python/voter/auto_voter.py
```

Wait for `ORACLE READY`. At each start, the oracle reconstructs its UTXO index **in memory**, one block at a time, over the required first 131,000 blocks. No Bitcoin cache is stored on disk. Keep each service's SQLite event database when restarting after an RPC failure.

5. Run both loan scenarios / project root

```
./.venv/bin/python python/simulate.py \
  --btc-address YOUR_VALIDATED_ADDRESS --principal 1
```

Choose an address from the validated summary. The demo checks a fresh oracle balance and contributes. Scenario A funds 1 ETH at 5%, repays two 0.525 ETH installments and claims gain. Scenario B funds 0.5 ETH, waits beyond its three-block term and requests compensation, recording default. On a fresh deployment the reserve pays 0.025 ETH; a zero-reserve retry preserves the remaining 0.475 ETH claim. Receipts and compensation invariants are checked; balances, deadlines and loan states are printed. Both proposals wait for real YES votes and 12 blocks.

Submission. Include source, tests, configuration, lockfile, scripts and report in one ZIP. Exclude dependencies, `.env`, queues and regenerable chain data; provide Bitcoin input under course rules. See the README for file, test and report-build instructions. Repository edits alone do not upgrade an existing deployment.